VISUAL SPEAKER NOTES • CONTENT-DRIVEN EDITION

REA: Reverse Engineer Anything

Morluto/rea — Reverse engineer anything with agents

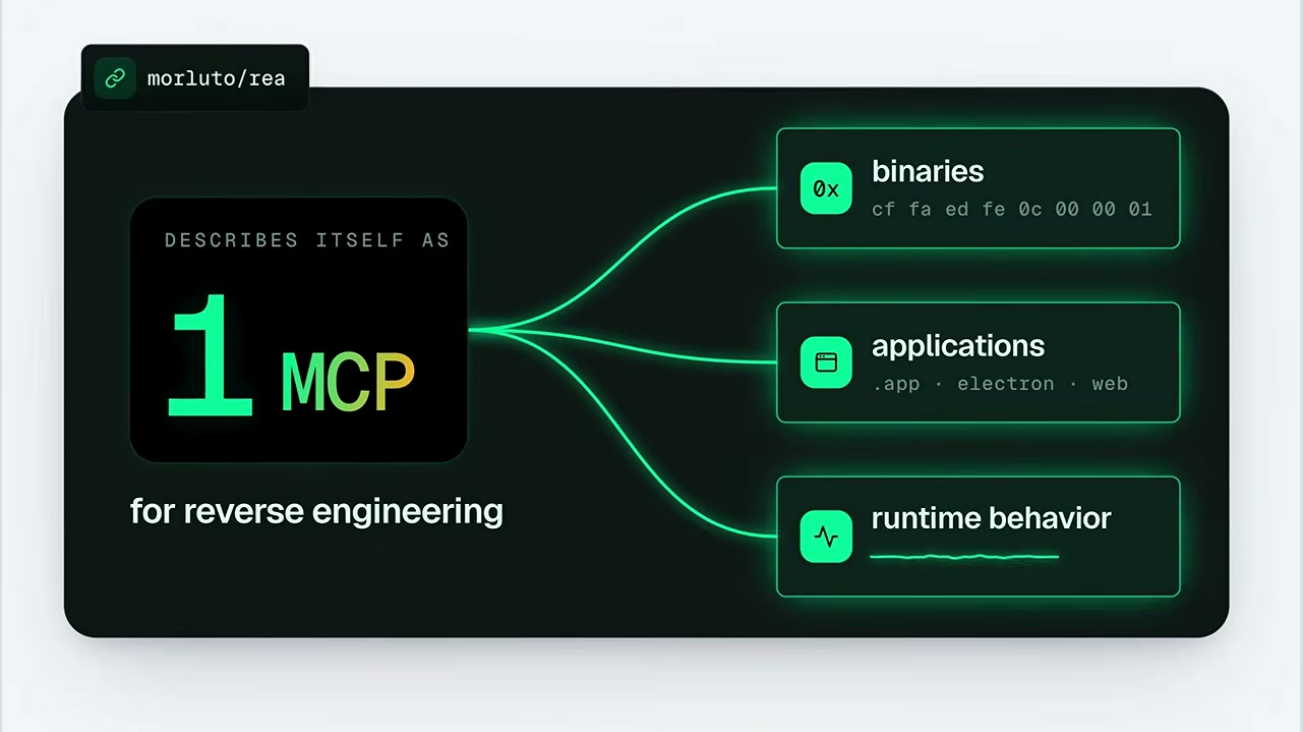
Signal Coders • 20261009 • 09:58

- 19 substantive visual(s), selected by content
- Repeated builds, presenter-only shots, and filler excluded
- Output length expands or contracts with the source material
- Speaker claims and caveats belong in the notes

https://www.youtube.com/watch?v=U_8af4oJpMo

Your Agent Gets a Decompiler

REA describes one MCP layer spanning binaries, applications, and runtime behavior.



SPEAKER NOTES

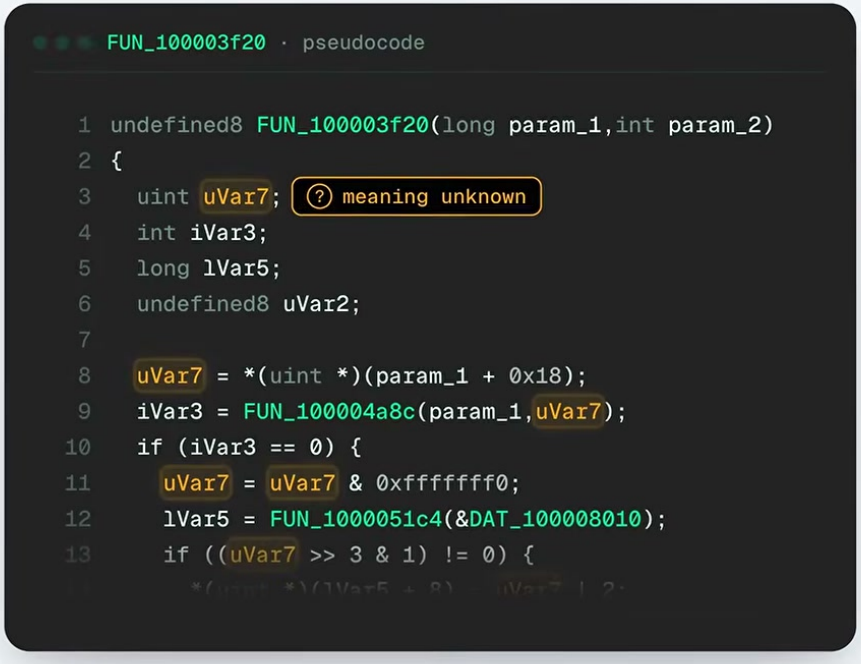
- REA is short for Reverse Engineer Anything and is published at github.com/morluto/rea.
- The project presents one MCP server for reverse engineering binaries, applications, and runtime behavior.
- The advertised workflow asks an agent to inspect an app without source, explain a feature, show evidence, and help recreate it.

WHY IT MATTERS

It moves reverse-engineering tools into the same interface coding agents already use while making evidence part of the requested output.

Why uVar7 Used To Cost Days

Traditional decompiler output leaves analysts to reconstruct meaning from stripped pseudocode.



SPEAKER NOTES

- Stripped binaries are opened in specialist tools such as Ghidra, IDA, or Hopper.
- Decompiler output may contain generic names such as uVar7 because original names and types are gone.
- Analysts traditionally spend days building a mental model that is difficult for others to inspect.

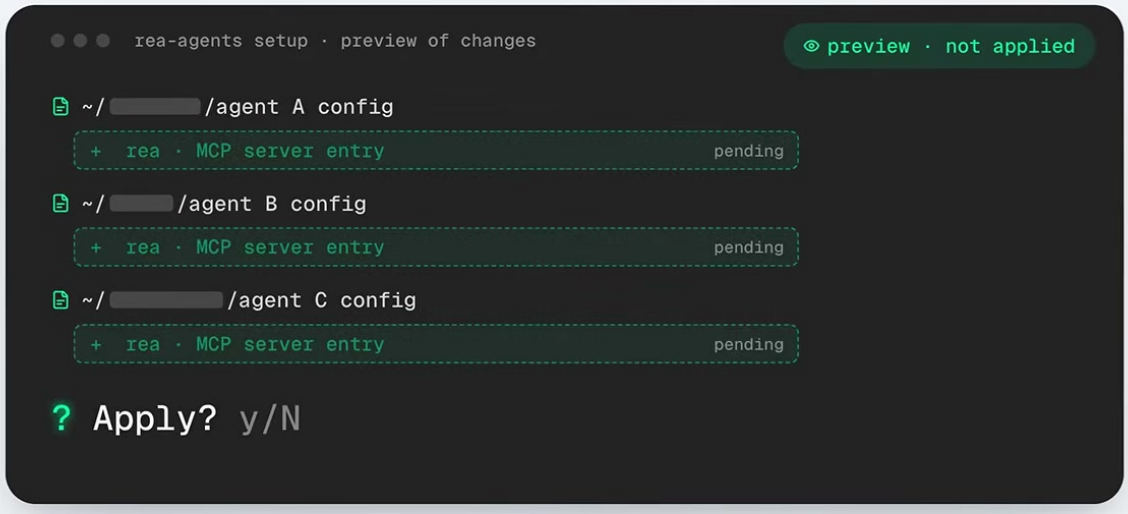
WHY IT MATTERS

REA targets the costly interpretation layer around decompilers, not the underlying hard problem of decompilation itself.

Source: https://www.youtube.com/watch?v=U_8af4oJpMo

What Setup Actually Changes

The setup flow previews configuration edits and asks before applying them.



SPEAKER NOTES

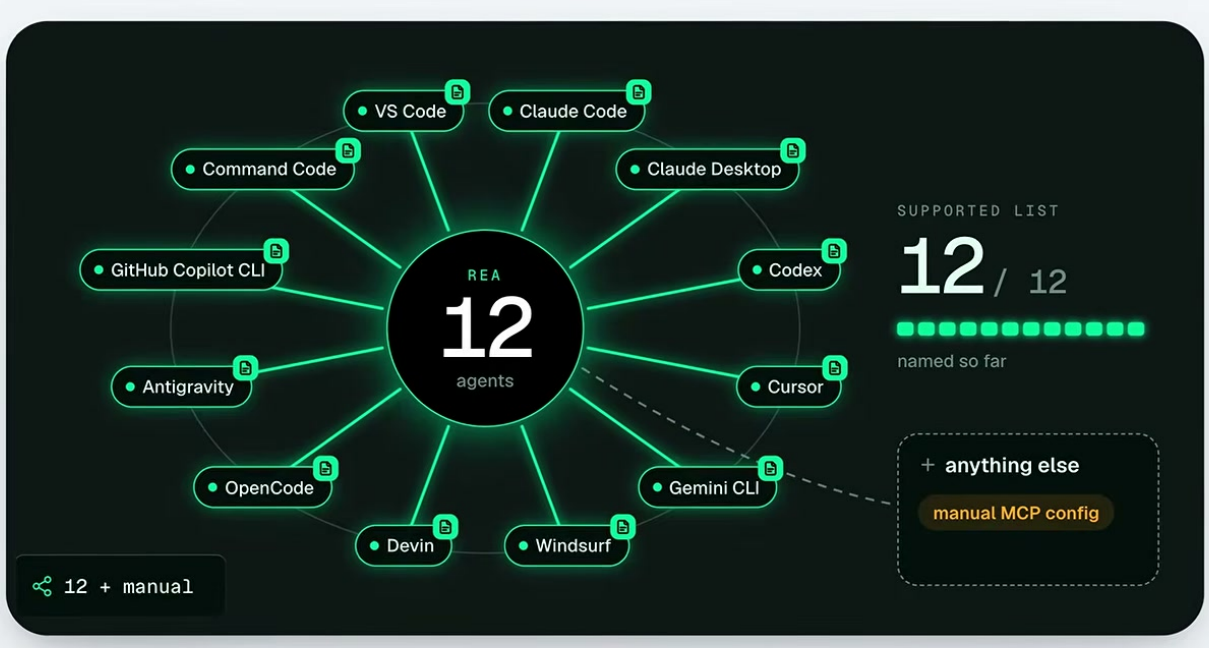
- npx rea-agents setup shows the exact paths and proposed changes before writing configuration.
- Existing configuration is backed up first, and the preview remains unapplied until the user confirms.
- Already registered agents are preselected; newly detected agents are listed but not selected merely because they were found.

WHY IT MATTERS

A transparent, consent-based setup reduces the risk of silently altering multiple agent configurations.

The Twelve Supported Agents

One server is registered across twelve named agent clients, with manual MCP configuration for others.



SPEAKER NOTES

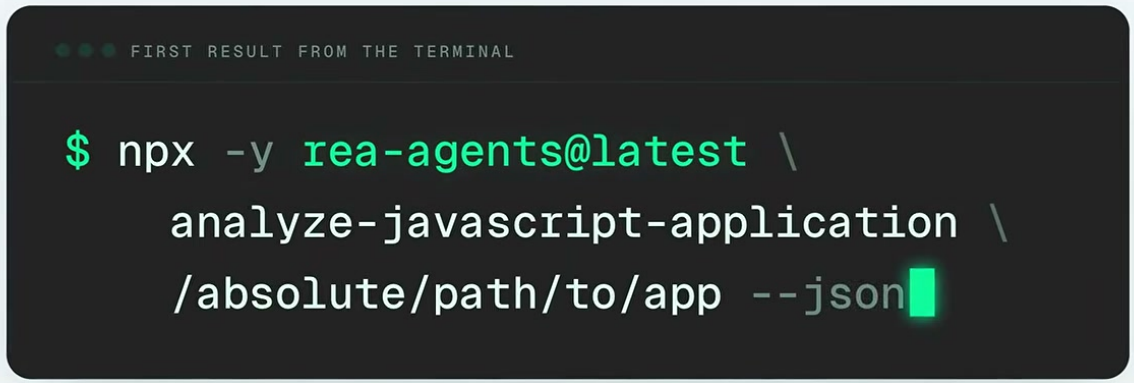
- The documented list contains twelve clients, including Claude Code, Claude Desktop, Codex, Cursor, Gemini CLI, and VS Code.
- Setup gives each chosen client MCP access plus REA's guided investigation workflow.
- Unsupported clients can use a manual MCP configuration path rather than a separate plugin.

WHY IT MATTERS

The integration is centralized: one server and workflow can be reused across many coding-agent environments.

The No-Setup JavaScript Command

An extracted Electron or JavaScript tree can be analyzed directly from one NPX command.



SPEAKER NOTES

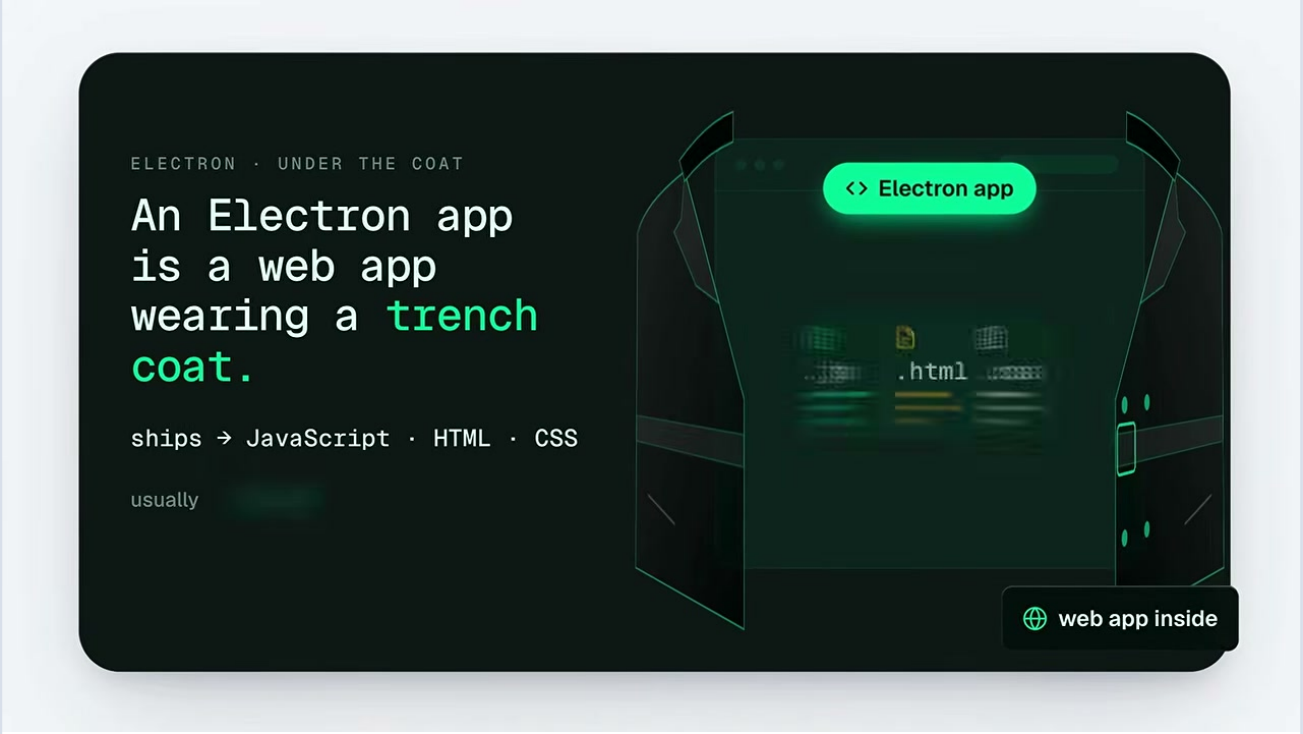
- The command is npx -y rea-agents@latest analyze-javascript-application [absolute-path] --json.
- It accepts an extracted JavaScript or Electron application tree, or an ASAR archive.
- This path requires no MCP configuration, no Ghidra or Hopper, and does not execute the target application.

WHY IT MATTERS

It provides a low-friction way to test the documented workflow before installing or configuring native-analysis tooling.

Electron Is A Web App

Its JavaScript, HTML, and CSS remain source text inside the package, even when bundled and minified.



SPEAKER NOTES

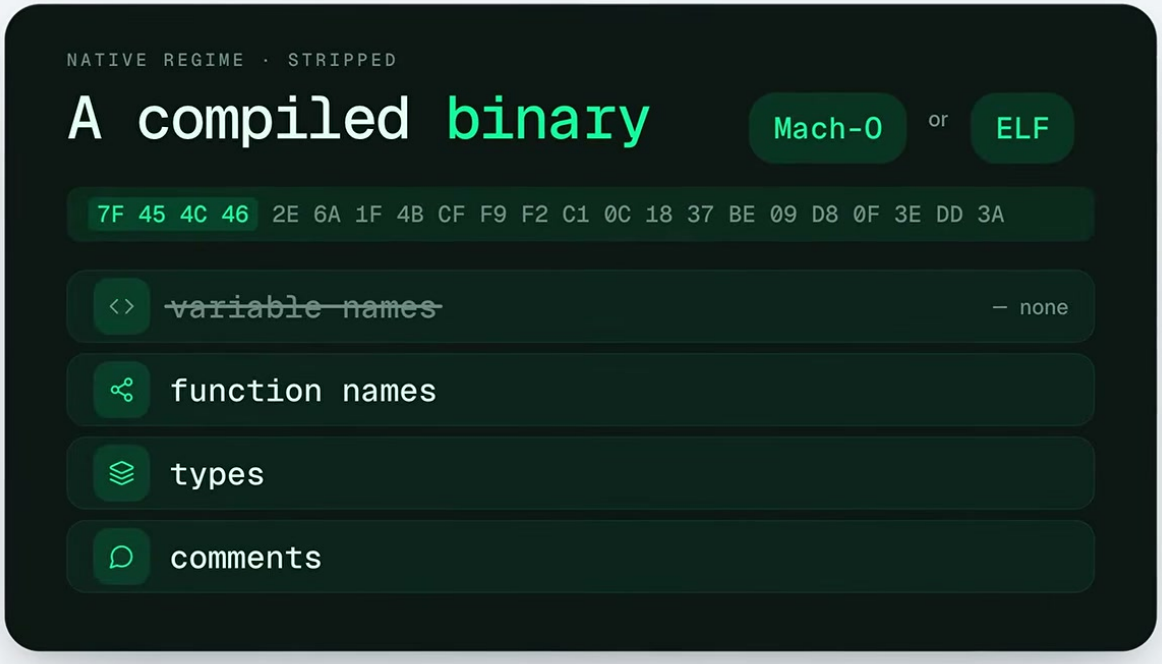
- The talk describes Electron as a web app wearing a trench coat.
- ASAR is presented as a concatenated file blob with a header index, not encryption.
- Static analysis can parse modules, rebuild call relationships, and recover a graph without executing code.

WHY IT MATTERS

Electron is the easy regime because packaging and minification obscure structure without compiling the logic into machine code.

Native Binaries Are The Hard Regime

Compiled Mach-O and ELF files lose the source-level information static JavaScript analysis retains.



SPEAKER NOTES

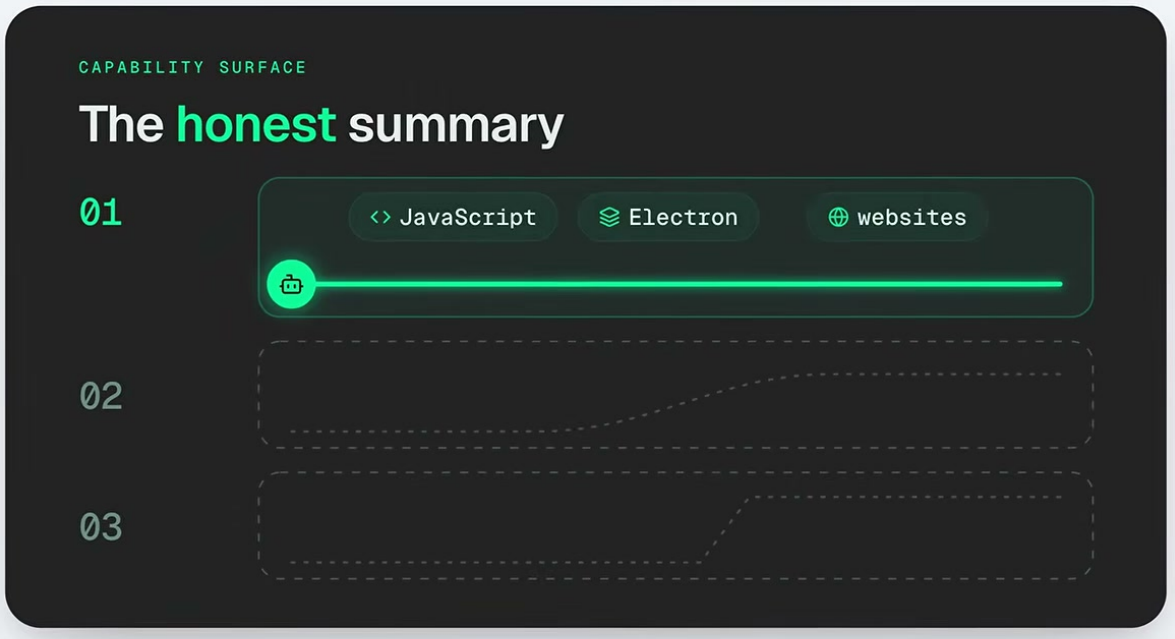
- Compiled native binaries may lack variable names, function names, types, and comments.
- A decompiler must lift machine instructions, reconstruct control flow, infer types, and emit C-like pseudocode.
- REA does not reimplement decompilation; native analysis requires an installed Hopper, Ghidra, or IDA engine.

WHY IT MATTERS

The native path depends on serious decompiler software already present on the machine, so it has different prerequisites from JavaScript analysis.

Where REA Is Frictionless

JavaScript, Electron, and websites are the lowest-friction path; native targets depend on a local engine.



SPEAKER NOTES

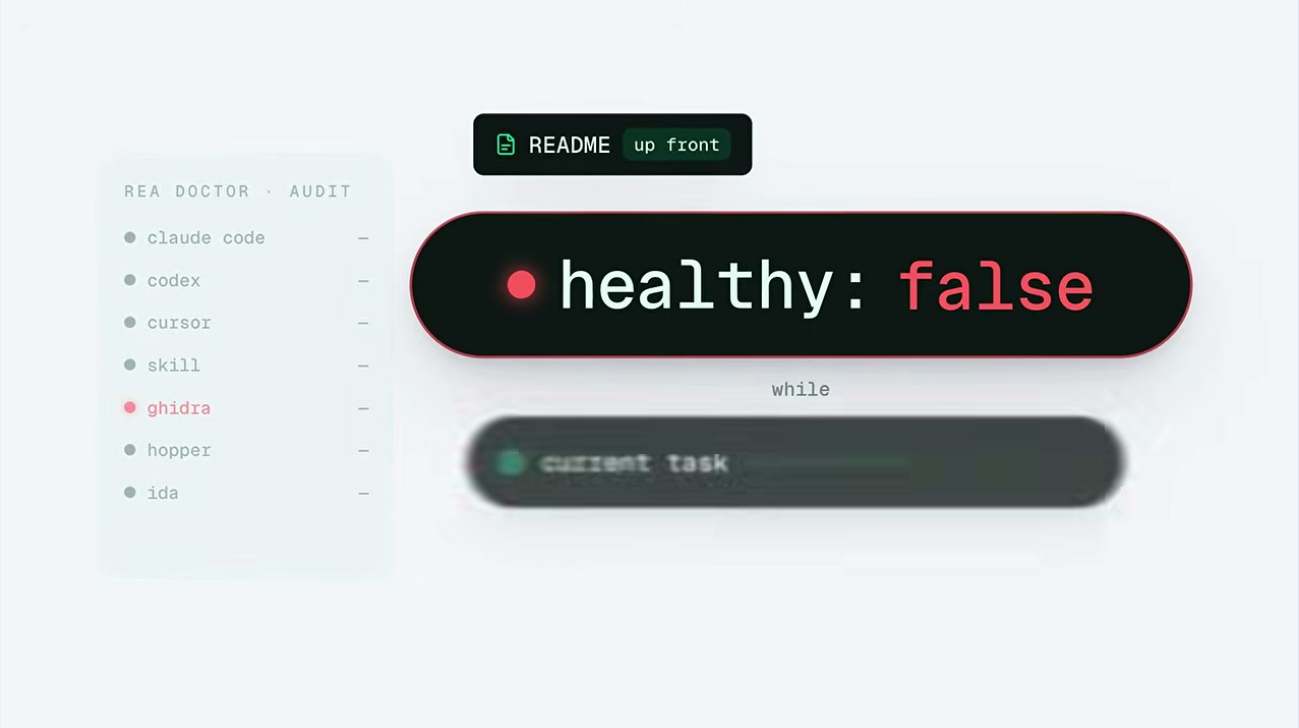
- JavaScript, Electron, and website analysis are presented as the frictionless lane.
- .NET is a middle case because intermediate language preserves more structure than raw machine code.
- Analysis runs locally; the talk says the target binary is not uploaded to make the workflow operate.

WHY IT MATTERS

Choosing the target type first determines whether REA can work directly or needs a separately installed decompiler.

rea doctor And Healthy False

A full audit can be unhealthy even when the scoped task is ready to run.



SPEAKER NOTES

- rea doctor without options audits detected agent registrations, the installed skill, and optional analysis engines.
- A missing Ghidra installation is a real audit finding but may be irrelevant to an Electron analysis.
- Therefore healthy: false does not by itself mean the current scoped task cannot work.

WHY IT MATTERS

Tool health must be interpreted against the task's actual dependencies rather than treated as one universal green light.

Scoped Readiness, Not Green Lights

Explicit scope checks determine health and exit status; unrelated findings remain informational.



SPEAKER NOTES

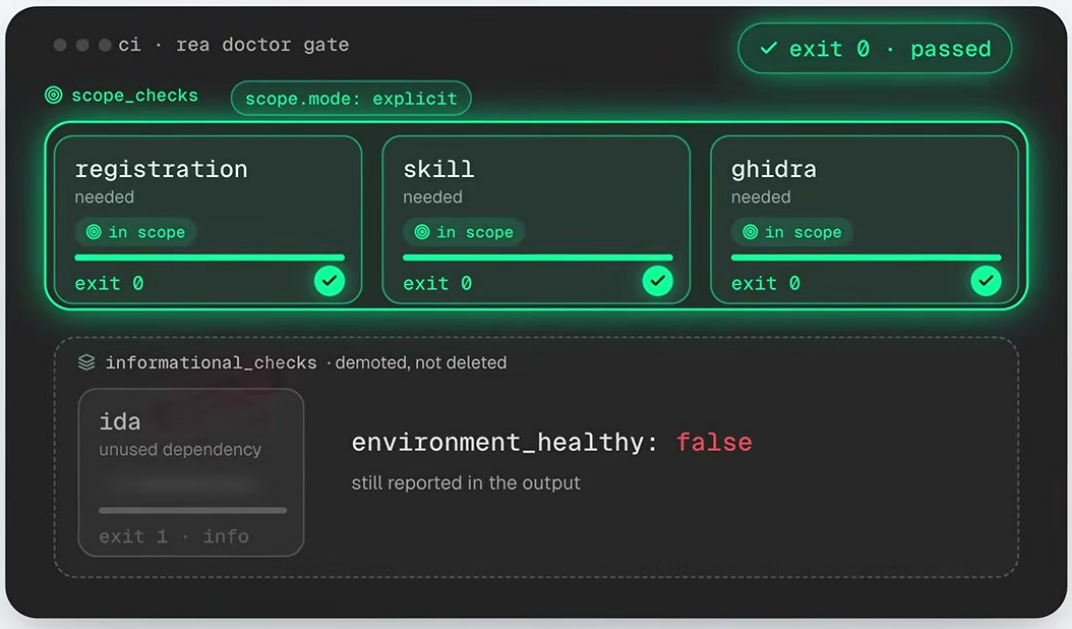
- Provider, client, and skill options can scope rea doctor to the dependency being troubleshot.
- With scope.mode explicit, only scope_checks determine healthy and the process exit status.
- Other findings move to informational_checks, while environment_healthy still summarizes the full audit.

WHY IT MATTERS

Scoped readiness preserves visibility without blocking work on dependencies the task does not use.

Why CI Pipelines Get Ignored

Unused dependencies can stay visible without causing the scoped gate to fail.



SPEAKER NOTES

- Global failures caused by dependencies a job does not use train teams to ignore CI signals.
- The corrected gate passes its registration, skill, and selected-provider checks while demoting an unused engine to informational.
- environment_healthy may remain false even when the scoped command exits successfully.

WHY IT MATTERS

A gate is credible only when failure means the current job cannot proceed.

REA Refuses To Guess Engines

One native input with two eligible engines is treated as an ambiguity, not an invitation to pick silently.



SPEAKER NOTES

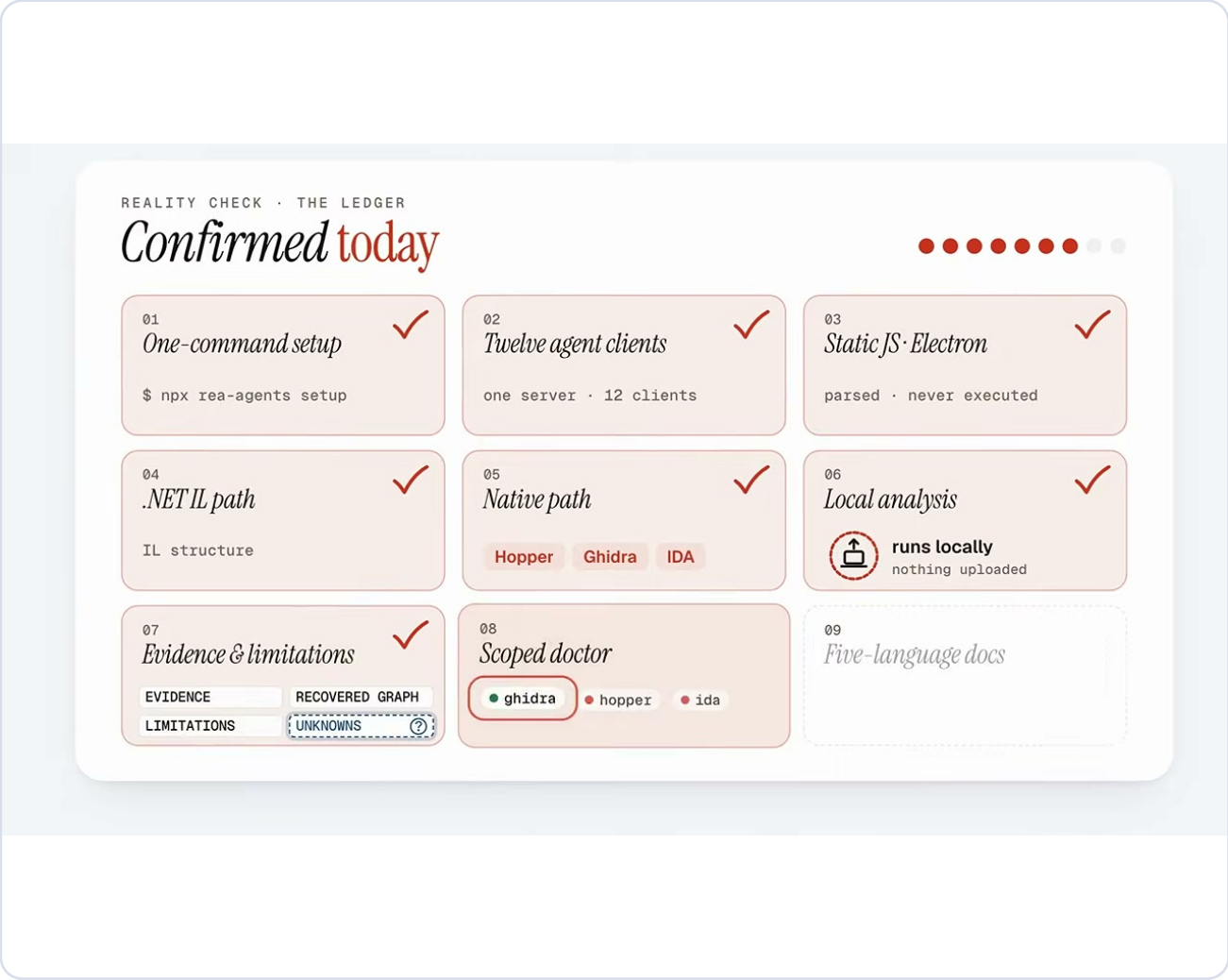
- When multiple installed engines support a native target, REA stops with capability_unavailable.
- The reported selection_reason is ambiguous, so the user must choose an engine.
- Different decompilers can produce meaningfully different pseudocode; filesystem ordering must not determine the evidence trail.

WHY IT MATTERS

Stopping instead of guessing keeps the analysis path intentional and reproducible.

What The Repo Confirms Today

The documented capability ledger separates current features from unsupported inference.



SPEAKER NOTES

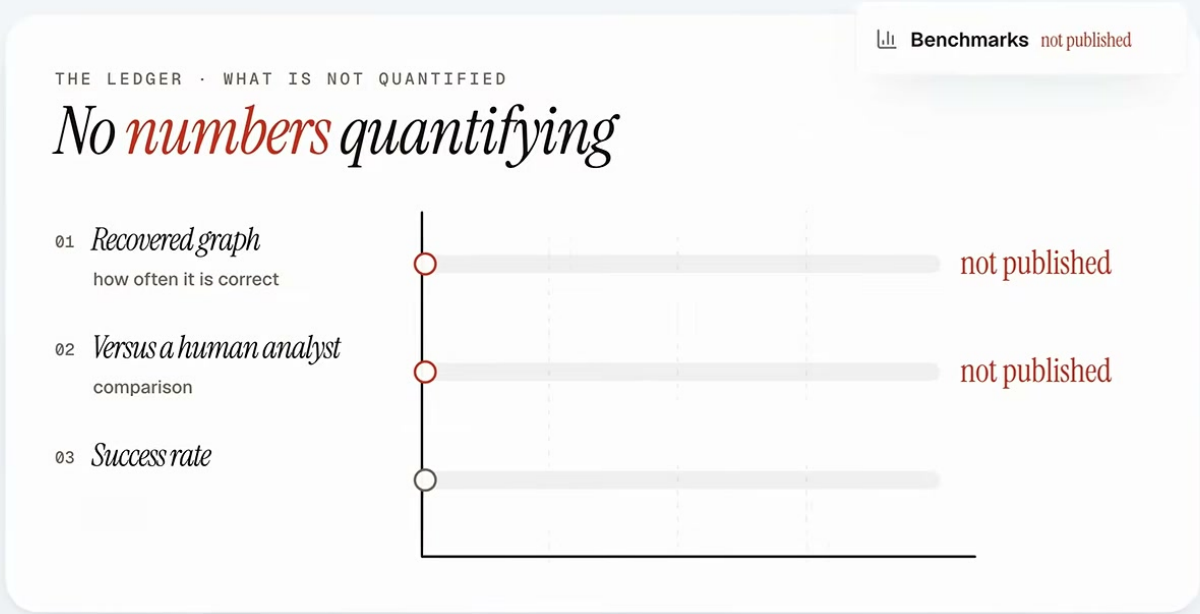
- The repository documents the MCP server, twelve-client setup, consent and configuration backups, and terminal commands.
- It documents static JavaScript and Electron analysis without an engine or execution, plus native analysis through Hopper, Ghidra, or IDA.
- It also publishes an investigation model, tool catalog, roadmap, guides, showcases, and community channels.

WHY IT MATTERS

The project is presented as a documented workflow with evidence and limitations, not merely a one-file wrapper.

No Accuracy Benchmarks Published

The repository material does not quantify correctness or success rates.



SPEAKER NOTES

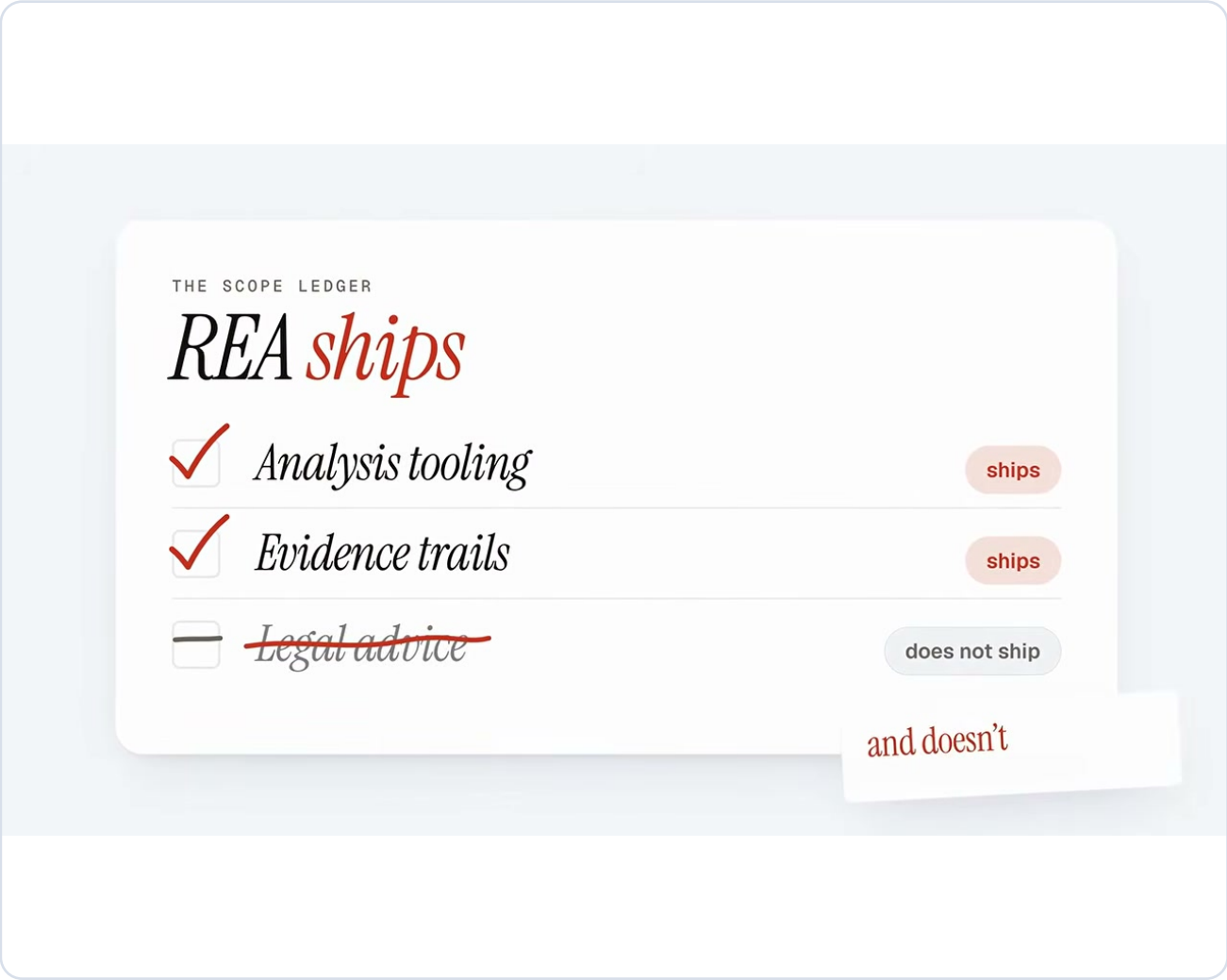
- No published numbers quantify how often a recovered graph is correct.
- No comparison with a human analyst or measured success rate is reported.
- Performance on obfuscated or packed bundles—including string arrays, control-flow flattening, and virtualized interpreters—is not stated.

WHY IT MATTERS

The documented feature set should not be mistaken for measured analytical accuracy.

Authorization Is Outside The Tool

REA ships analysis tooling and evidence trails, not permission or legal advice.



SPEAKER NOTES

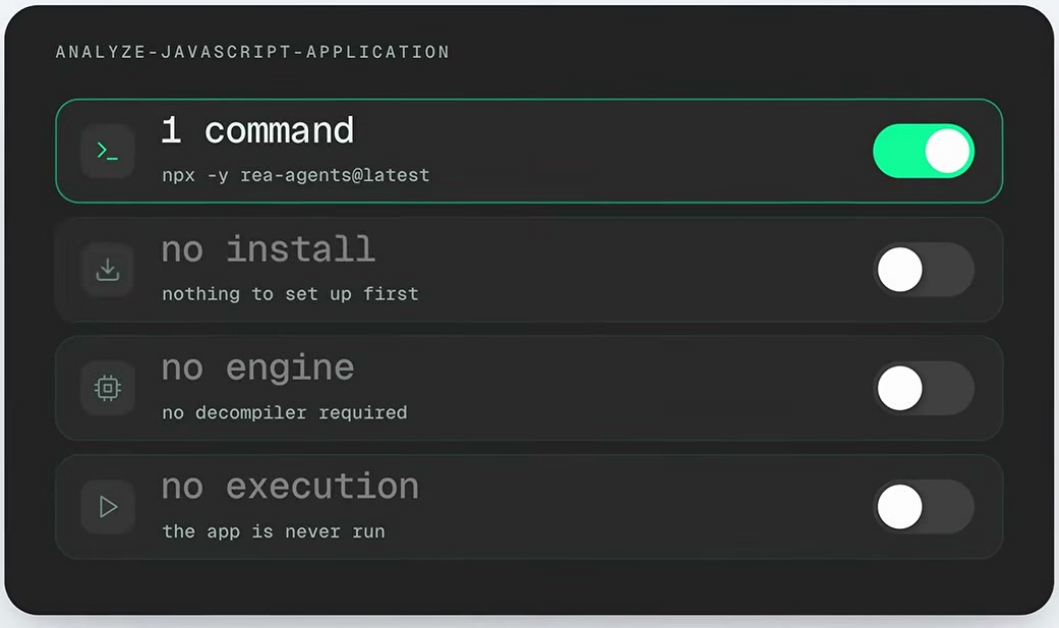
- Whether a target may be analyzed depends on its licence agreement and the applicable jurisdiction.
- The clean-room convention separates the team reading the original from the team implementing from a written specification.
- The talk notes a structural concern when the same agent reads the binary and writes the implementation; REA does not provide legal advice.

WHY IT MATTERS

Technical capability does not establish authorization, so users must resolve licence and jurisdiction questions independently.

Your Cheapest Experiment This Week

Try the static JavaScript analyzer on an Electron app you already have.



SPEAKER NOTES

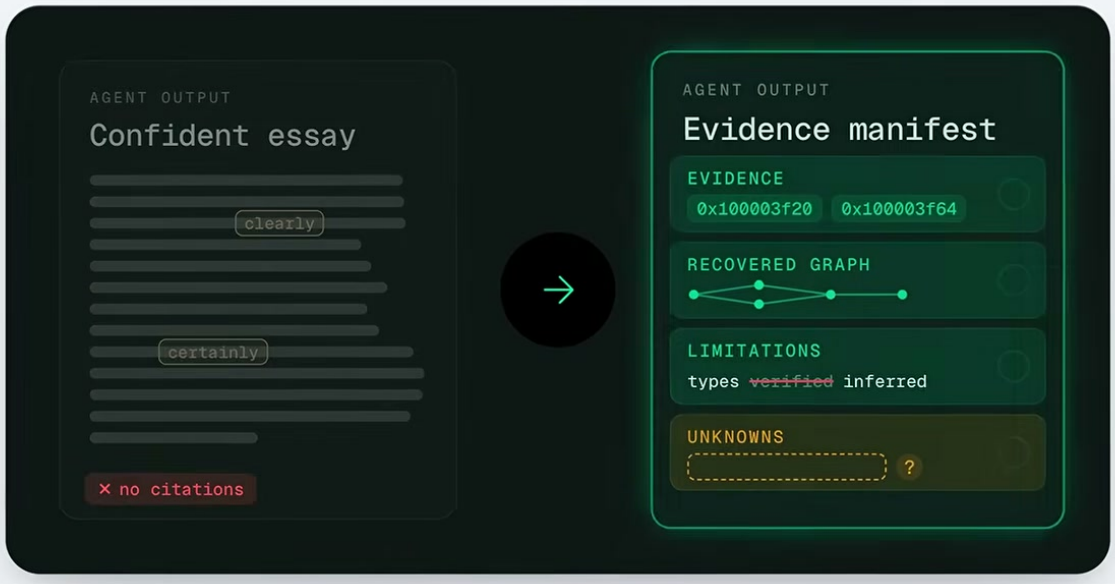
- Find an installed Electron app's directory or ASAR archive and pass its absolute path to analyze-javascript-application.
- The experiment is described as one command with no prior install, no decompiler engine, and no target execution.
- The result is a structured map intended to help answer how a shipping feature is assembled.

WHY IT MATTERS

The suggested first test isolates REA's lowest-friction capability before committing to broader setup.

Evidence, Graph, Limitations, Unknowns

Structured output replaces a confident essay with an auditable manifest.



SPEAKER NOTES

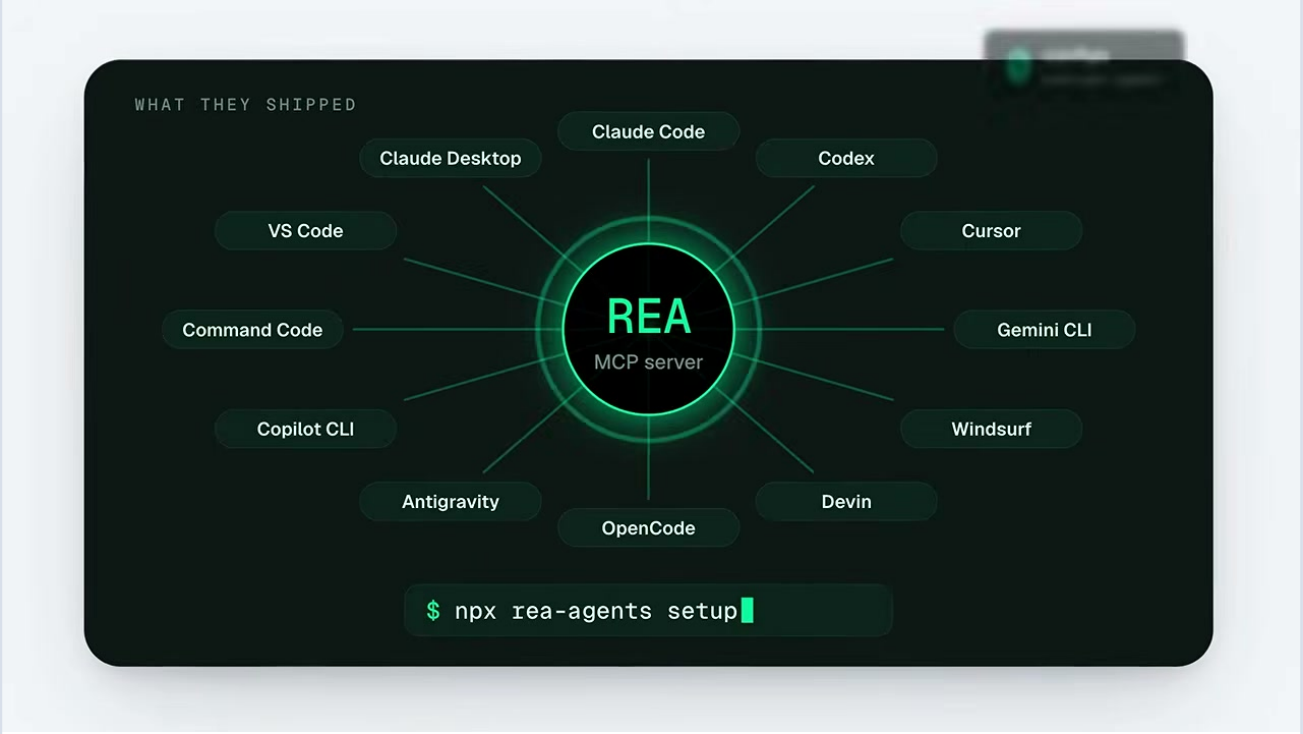
- The output shape separates evidence references, a recovered graph, limitations, and unknowns.
- The slide contrasts that manifest with prose that sounds certain but provides no citations.
- Naming limitations and unknowns shows reviewers where the analysis remains inferred or unresolved.

WHY IT MATTERS

An answer that exposes its evidence and uncertainty is easier to audit than fluent unsupported prose.

One Server, Twelve Agents

The same MCP tool catalog is registered across twelve supported clients.



SPEAKER NOTES

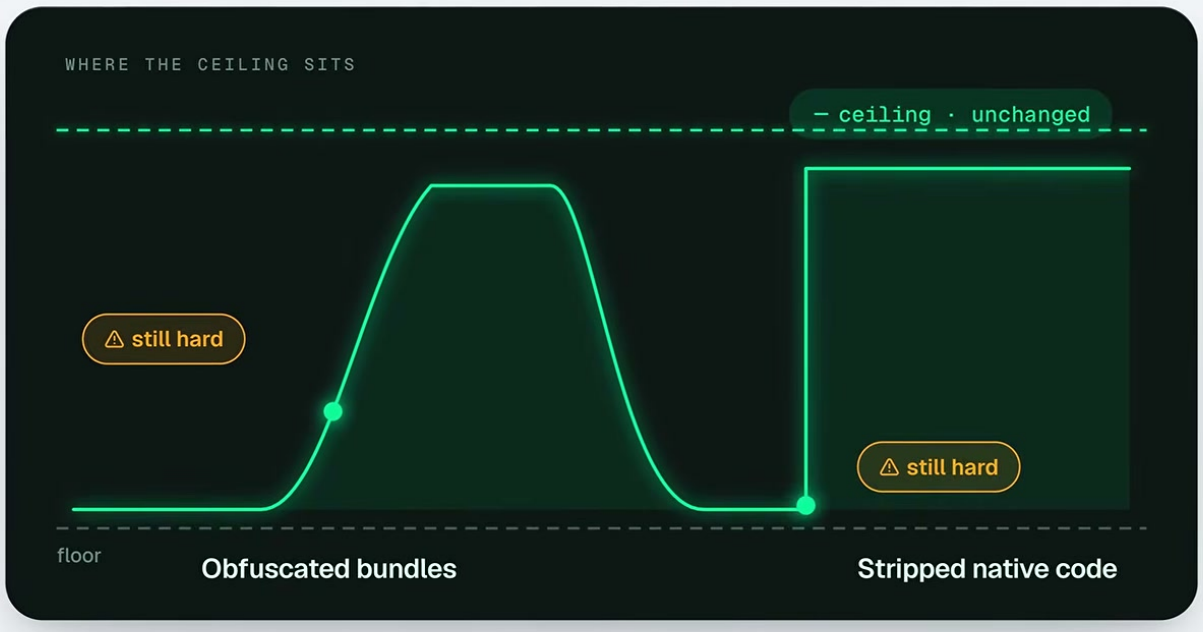
- REA uses one MCP server rather than twelve separate plugins.
- The setup command writes the selected client configurations after showing the paths it will touch.
- The same tool catalog and investigation workflow are available across the supported agent environments.

WHY IT MATTERS

Interoperability becomes configuration work instead of repeated integration development.

The Floor Moved, Not The Ceiling

Obfuscated bundles and stripped native code remain hard even as the starting point improves.



SPEAKER NOTES

- The talk says the skill ceiling has not moved: obfuscated bundles and stripped native code are still hard.
- Compiled targets still require a real decompiler installed on disk.
- What changes is the floor: initial exploration that once meant days of deciphering generic variables can begin as an agent prompt with citations.

WHY IT MATTERS

REA lowers the cost of starting an investigation without claiming to solve the hardest cases.